ASSIGNMENT – 2

C++ FUNCTIONS AND OBJECT-ORIENTED PROGRAMMING

Submitted by: J Roopa Sai Divya

Register Number: 192524140

Course: C++ / DSA

Assignment Coverage: CO2

Submission Date: 31/10/2026

GitHub Repository Link: [https://github.com/roopajallu814-arch/C-DSA0104-Roopa-192524140-]

CONTENTS

Q1 – Call by Value, Reference and Pointer

Q2 – Return by Reference and Its Dangers

Q3 – Default Arguments and Function Overloading

Q4 – Class with an Array Inside It

Q5 – Static Members and Arrays of Objects

Q6 – Friend Functions and Inline Functions

Q7 – Diagnose, Redesign and Validate

GitHub Submission Checklist

Q1 – Call by Value, by Reference and by Pointer

1.1 Aim

To compare call by value, call by reference and call by pointer, and choose a safe interface for updating a hostel fee.

1.2 Program

```
#include <iostream>
using namespace std;

void changeByValue(int a) { a += 10; }
void changeByReference(int& b) { b += 10; }
void changeByPointer(int* c) { if (c != nullptr) *c += 10; }

int& bigger(int& p, int& q) { return (p > q) ? p : q; }
void show(int x, int y = 5, int z = 10) {
    cout << x + y + z << ' ';
}

int main() {
    int a = 1, b = 2, c = 3;
    changeByValue(a);
    changeByReference(b);
    changeByPointer(&c);
    cout << a << ' ' << b << ' ' << c << '\n';

    bigger(a, c) = 100;
    cout << a << ' ' << c << '\n';

    show(1);
    show(1, 2);
    show(1, 2, 3);
    cout << '\n';
    return 0;
}
```

1.3 Predicted and observed output

Predicted output (verify by compiling and running the code):

```
1 12 13
100 100
16 13 6
```

Explanation: changeByValue receives a copy, so the original a remains 1. The reference parameter b aliases the caller's b, so b becomes 12. The pointer c points to the caller's variable, so its value becomes 13. bigger(a, c) returns a reference to the larger variable; at that point c is larger than a, so assigning 100 changes c. The

default arguments in show(1) are y=5 and z=10; show(1,2) uses z=10; show(1,2,3) uses all supplied values.

1.4 Complete the code – concession by reference

```
#include <iostream>
using namespace std;

void applyByValue(long long fee, int percent) {
    fee -= fee * percent / 100;
}
void applyConcession(long long& fee, int percent) {
    fee -= fee * percent / 100;
}
int main() {
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    cout << "after call by value : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    cout << "after call by reference: " << hostelFee << '\n';
    return 0;
}
```

Expected output:

```
after call by value : 45000
after call by reference: 40500
```

Call by value leaves the stored fee unchanged because only a local copy is reduced. Call by reference changes the original variable directly.

1.5 Alternative design and complexity

```
long long applyConcession(long long fee, int percent) {
    return fee - fee * percent / 100;
}
// Usage:
hostelFee = applyConcession(hostelFee, 10);
```

Returning the new fee makes the update explicit and avoids hidden side effects. For a single long long, value copying is small. For a large Student object, passing by value copies the object, while reference and const reference avoid the full copy. Time is O(1) for this arithmetic in all versions. Use const T& when the function must read a potentially large object without modifying it. For several concessions, returning the new fee is a clear choice when the caller should decide whether to

store the result; reference is suitable when in-place modification is the intended contract.

Q2 – Return by Reference and Its Dangers

2.1 Program

```
#include <iostream>
using namespace std;

int& highestMark(int marks[], int n) {
    int index = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[index]) index = i;
    return marks[index];
}

// Incorrect: do not use this function.
int& badReference() {
    int local = 10;
    return local; // dangling reference; compiler warning expected
}

int main() {
    int marks[] = {78, 91, 84, 88};
    highestMark(marks, 4) += 2;
    cout << "Highest mark after moderation: " << highestMark(marks, 4) << '\n';
    return 0;
}
```

Expected output: Highest mark after moderation: 93

2.2 Explanation

highestMark returns a reference to an existing array element, so the caller can update it and use the function call on the left side of an assignment. Returning a reference to a local variable is invalid because the local object is destroyed when the function returns. The reference then dangles, and using it causes undefined behaviour. A compiler commonly warns about returning the address/reference of a local variable.

- T&: caller may modify the referred object.
- const T&: caller can read the object through the reference but cannot modify it through that reference.

- T returned by value: a separate copy is returned; changing the copy does not change the original array element.

If a std::vector grows and reallocates its storage, references and pointers to its elements can become invalid. An index remains a numeric position, but it must still be checked if elements are inserted, erased, or reordered.

2.3 Index and pointer alternatives; complexity

```
int highestIndex(const int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return best;
}
// Use: marks[highestIndex(marks, n)] += 2;

// Pointer alternative:
int* highestPointer(int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return &marks[best];
}
// Use: *highestPointer(marks, n) += 2;
```

Finding the maximum takes O(n) time and O(1) extra space for all three approaches. A reference and pointer can be invalidated by vector reallocation; an index is often easier to recompute and does not itself point to a memory address, though its logical meaning can change after edits.

Q3 – Default Arguments and Function Overloading

3.1 Why ambiguity occurs

If charge(double), charge(double, bool), and charge(double, bool=false, bool=false) are all declared, a call such as charge(2.0) can match all three. The compiler cannot select one unambiguously, so the program fails to compile.

3.2 Clear redesigned implementation

```
#include <iostream>
using namespace std;
```

```
struct Parcel {
    double weightKg;
    bool express;
    bool insured;
};

double charge(int grams) {
    return grams * 0.02;
}
double charge(double weightKg, bool express = false, bool insured = false) {
    double amount = weightKg * 50.0;
    if (express) amount += 30.0;
    if (insured) amount += 10.0;
    return amount;
}
double charge(const Parcel& p) {
    return charge(p.weightKg, p.express, p.insured);
}

int main() {
    cout << charge(500) << '\n';       // int overload: grams
    cout << charge(2.0) << '\n';       // double overload: kilograms
    cout << charge(2.0, true) << '\n'; // express
    cout << charge(2.0, true, true) << '\n';
    cout << charge(2) << '\n';
    cout << charge(2.0f) << '\n';      // float promotes to double
    Parcel p{1.5, false, true};
    cout << charge(p) << '\n';
    return 0;
}
```

For the chosen example rates, the calls select: int grams overload for charge(500) and charge(2); double overload for charge(2.0), charge(2.0,true), charge(2.0,true,true), and charge(2.0f); Parcel overload for charge(p). A float argument promotes to double. Note that charge(2) selects the int overload exactly, so units must be clearly documented.

3.3 Overload resolution and defaults

The compiler generally prefers an exact match, then a promotion, then a standard conversion. Default arguments fill omitted trailing parameters; they must be supplied from the right because a later parameter cannot be omitted while an earlier one is missing. Default arguments are not a replacement for distinct overloads when signatures overlap.

3.4 Alternative and complexity

A single Parcel structure can group related options and is easier to extend when new fields are added. A template can help when the input types follow a common rule, but it should not be used just to hide unclear units. Overload resolution occurs at compile time and adds no runtime cost. Passing a large Parcel by const reference avoids copying; a small structure can be passed by value. Each charge calculation is O(1).

Q4 – Specifying a Class with an Array Inside It

4.1 SavingsAccount implementation

```
#include <iostream>
using namespace std;

class SavingsAccount {
private:
    int accountNumber;
    double balance;
    double transactions[10];
    int count;

    void record(double amount) {
        if (count < 10) {
            transactions[count++] = amount;
        } else {
            for (int i = 1; i < 10; ++i)
                transactions[i - 1] = transactions[i];
            transactions[9] = amount;
        }
    }

public:
    SavingsAccount(int number, double openingBalance)
        : accountNumber(number), balance(openingBalance), count(0) {}

    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            record(amount);
        }
    }

    bool withdraw(double amount) {
        if (amount <= 0 || amount > balance) return false;
        balance -= amount;
        record(-amount);
        return true;
    }

    void miniStatement() const {
        cout << "Account: " << accountNumber << "\nBalance: " << balance
```

```
            << "\nRecent transactions:\n";
        for (int i = 0; i < count; ++i) cout << transactions[i] << '\n';
    }
};

int main() {
    SavingsAccount account(1925, 1000.0);
    account.deposit(250);
    account.withdraw(100);
    account.miniStatement();
    return 0;
}
```

4.2 Explanation and design decision

The array is private so external code cannot alter transaction history or break the class's rules. Public methods validate operations and maintain a consistent balance. In this implementation, deposits are positive transaction values and withdrawals are negative values.

When the 11th transaction arrives, shifting discards the oldest item and moves nine items left before appending the new transaction. This costs O(k), where k is the number of stored transactions. A circular buffer writes the new item at the next index and advances the start position, costing O(1) per insertion. Both designs use O(k) storage. If the history grows from 10 to 50, a fixed array needs its capacity and related loop bounds updated. A circular buffer can also be resized by changing the capacity, but its indexing logic must be tested carefully.

A std::deque can use push_back and pop_front to keep the most recent k entries with approximately O(1) end operations. It is convenient but has library/container overhead. For a fixed mini-statement size, a circular buffer is efficient and predictable.

Q5 – Static Members and Arrays of Objects

5.1 Static token program and predicted output

```
#include <iostream>
using namespace std;
```

```
class Token {
    static int next;
    int id;
public:
    Token() { id = ++next; }
    int get() const { return id; }
    static int issued() { return next - 100; }
};
int Token::next = 100;

int main() {
    Token counter[3];
    Token extra;
    cout << counter[0].get() << ' ' << counter[2].get() << ' '
         << extra.get() << ' ' << Token::issued() << '\n';
}
```

Predicted output:

```
101 103 104 4
```

The static data member next is shared by every Token and is defined once outside the class, initialized to 100. The three objects in counter and the extra object cause the constructor to run four times. Each constructor increments next before assigning id.

5.2 Complete BusTicket code

```
#include <iostream>
#include <string>
using namespace std;

class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    string name;
public:
    explicit BusTicket(const string& n) : seat(0), name(n) {
        if (issued < CAPACITY) {
            ++issued;
            seat = issued;
        }
    }

    static int count() { return issued; }

    void print() const {
        cout << name << ": "
             << (seat ? "Seat " + to_string(seat) : string("Waiting"))
             << '\n';
    }
};
int BusTicket::issued = 0;
```

```
int main() {
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
                         BusTicket("Chitra"), BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    cout << "Issued: " << BusTicket::count() << '\n';
    return 0;
}
```

Expected output:

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

5.3 Explanation and alternatives

issued and CAPACITY belong to the class, not to an individual ticket. A static member function can be called as BusTicket::count(); it has no this pointer and cannot directly access a particular object's non-static name or seat. An array of objects constructs each element, so the shared counter is updated for each constructor call. Calling a function twice creates new local objects on each call, but the static issued counter persists for the lifetime of the program; it does not reset automatically.

A global counter could work but would expose state outside the class and risk name conflicts. A TicketOffice object passed to each ticket would better support multiple buses or offices, each with its own capacity and count. Static counting is simple for one shared bus capacity. All approaches update/check a counter in O(1) time and use O(1) counter storage per counter instance.

Q6 – Friend Functions and Inline Functions

6.1 Completed code

```
#include <iostream>
#include <cstdlib>
using namespace std;
```

```
class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};

int minutesBetween(const Time& a, const Time& b) {
    int first = a.h * 60 + a.m;
    int second = b.h * 60 + b.m;
    return abs(first - second);
}

bool earlier(const Time& a, const Time& b) {
    if (a.h != b.h) return a.h < b.h;
    return a.m < b.m;
}

int main() {
    Time departure(9, 45), arrival(13, 10);
    cout << minutesBetween(departure, arrival) << ' '
         << boolalpha << earlier(arrival, departure) << '\n';
    return 0;
}
```

Expected output: 205 false

6.2 Tests and explanation

9:45 is 585 minutes after midnight; 13:10 is 790 minutes after midnight. Their absolute difference is 205 minutes. arrival is not earlier than departure, so earlier(arrival, departure) returns false. Test the same time twice: the difference should be 0 and earlier(t, t) should be false. Test 11:50 and 12:10: the difference should be 20 and 11:50 should be earlier.

A friend function is not a member, so it is called as minutesBetween(a, b), not a.minutesBetween(b). It can access private members because the class explicitly grants friendship. A member function can access private data through this and can be called as a.minutesTo(b). Public getters preserve private data but expose read access. Keep private fields private rather than making them public.

6.3 Inline and timing analysis

Defining a small function inside a class definition makes it implicitly inline. The inline keyword permits multiple identical definitions under the One Definition

Rule and suggests that the compiler may substitute the function body at the call site; it does not guarantee substitution. Compilers may inline functions automatically.

For a required performance measurement, compile equivalent benchmark programs with g++ -std=c++17 -O0 and g++ -std=c++17 -O2, run the same 10,000,000-call workload several times, and record the actual elapsed time. Do not invent timing values: optimization level, hardware, compiler and benchmark design affect results. At -O2, the compiler may inline and optimize calls; at -O0, call overhead may be more visible. The operations remain O(1) per call. Friend functions are justified when an operation treats two Time objects symmetrically; otherwise member functions or getters may provide a simpler interface.

Q7 – Diagnose, Redesign and Validate

7.1 Selected limitation

Selected program: Q4 SavingsAccount transaction history. The original fixed-array design shifts all stored transactions whenever the history is full. At a capacity of k, each new transaction after the buffer fills performs k−1 moves, so insertion is O(k). This becomes more work when the required history grows.

7.2 Measurable objective

Redesign the history so that adding a transaction takes O(1) time while retaining only the newest 50 transactions and preserving their chronological display order.

7.3 Revised design – circular buffer

```
// Pseudocode for a circular history buffer:
capacity = 50
array[capacity]
start = 0
count = 0

record(value):
    if count < capacity:
        array[(start + count) % capacity] = value
        count = count + 1
    else:
        array[start] = value
```

```
        start = (start + 1) % capacity

display:
    for i from 0 to count - 1:
        print array[(start + i) % capacity]
```

The circular buffer uses a fixed array of 50 values plus start and count indices. It requires O(50) storage, which is O(k) in terms of capacity, and each insertion uses O(1) time. It avoids shifting. The trade-off is more complicated indexing and a need to test wrap-around behaviour. A shifting array is simpler and can be preferable for very small histories.

7.4 Prediction before implementation

Prediction: once the buffer is full, the circular-buffer version should perform a constant number of index calculations and one write per insertion, whereas the shifting version moves up to 49 existing elements for each new transaction.

7.5 Validation plan

Run both versions with identical transaction sequences of 10, 50, 51, 1000 and 100000 entries. Confirm that both retain the newest 50 values in the same chronological order. Measure elapsed time over repeated runs and report the actual results below; do not fill this table with invented measurements.

Input transactions	Shifting time	Circular-buffer time	Correct newest 50?
10	Record actual result	Record actual result	Yes/No after test
50	Record actual result	Record actual result	Yes/No after test
51	Record actual result	Record actual result	Yes/No after test
1,000	Record actual result	Record actual result	Yes/No after test
100,000	Record actual result	Record actual result	Yes/No after test

7.6 Final evaluation

The redesign meets the objective if tests confirm identical retained history and the measured insertion time scales better as the history grows. The circular buffer reduces insertion work but makes display and indexing more complex. The

conclusion must be based on the measured results collected on the actual compiler and machine.

GitHub Submission Checklist

- Create a repository named DSA01-192524140-Assignment2.
- Create one folder per task: Task01 through Task07 (or Q1 through Q7).
- Add each C++ source file, relevant input files and output screenshots.
- Add README.md with name, register number, compiler/version, exact compile command, run steps and a short result note for every task.
- Commit work progressively, with at least one commit per task.
- Run each program yourself and capture genuine compiler/output evidence.
- Paste the working GitHub URL on the cover page and in the assignment submission field.
- Replace all evidence placeholders with your own actual output screenshots and measured timings before submission.

Suggested compile command: g++ -std=c++17 -Wall -Wextra task.cpp -o task

GitHub repository link: https://github.com/roopajallu814-arch/C-DSA0104-Roopa-192524140-. Confirm that the repository contains the assignment's required folders, source files, output screenshots, README.md, and progressive commits before submission.

Program Execution Screenshots

The following screenshots show the source code and the output produced by compiling and running the examples with g++ -std=c++17 -Wall -Wextra. Replace or supplement these with screenshots captured on your own computer if your faculty requires direct IDE evidence.

Q1 Call By Reference

C++ PROGRAM / OUTPUT SCREENSHOT — Q1 CALL BY REFERENCE

SOURCE CODE

```
#include <iostream>
using namespace std;
void changeByValue(int a) { a += 10; }
void changeByReference(int& b) { b += 10; }
void changeByPointer(int* c) { if (c != nullptr) *c += 10; }
int& bigger(int& p, int& q) { return (p > q) ? p : q; }
void show(int x, int y = 5, int z = 10) { cout << x + y + z << ' '; }
int main() {
 int a=1, b=2, c=3;
 changeByValue(a); changeByReference(b); changeByPointer(&c);
 cout << a << ' ' << b << ' ' << c << '\\n';
 bigger(a,c)=100;
 cout << a << ' ' << c << '\\n';
 show(1); show(1,2); show(1,2,3); cout << '\\n';
}
```

TERMINAL OUTPUT

```
1 12 13236621 1002366216 13 6 23662
```

Compile: SUCCESS | Run: SUCCESS

Execution status: Compile: SUCCESS | Run: SUCCESS

Q1 Concession

C++ PROGRAM / OUTPUT SCREENSHOT — Q1 CONCESSION

SOURCE CODE

```
#include <iostream>
using namespace std;
void applyByValue(long long fee, int percent) {
 fee -= fee * percent / 100;
}
void applyConcession(long long& fee, int percent) {
 fee -= fee * percent / 100;
}
int main() {
 long long hostelFee=45000;
 applyByValue(hostelFee,10);
 cout << "after call by value : " << hostelFee << '\\n';
 applyConcession(hostelFee,10);
 cout << "after call by reference: " << hostelFee << '\\n';
}
```

TERMINAL OUTPUT

```
after call by value : 4500023662after call by reference: 4050023662
```

Compile: SUCCESS | Run: SUCCESS

Execution status: Compile: SUCCESS | Run: SUCCESS

Q2 Highest Mark

C++ PROGRAM / OUTPUT SCREENSHOT — Q2 HIGHEST MARK

SOURCE CODE

```
#include <iostream>
using namespace std;
int& highestMark(int marks[], int n) {
 int index=0;
 for(int i=1;i<n;++i) if(marks[i]>marks[index]) index=i;
 return marks[index];
}
int main() {
 int marks[]={78,91,84,88};
 highestMark(marks,4)+=2;
 cout << "Highest mark after moderation: " << highestMark(marks,4) << '\\n';
}
```

TERMINAL OUTPUT

```
Highest mark after moderation: 9323662
```

Compile: SUCCESS | Run: SUCCESS

Execution status: Compile: SUCCESS | Run: SUCCESS

Q3 Overloading

C++ PROGRAM / OUTPUT SCREENSHOT — Q3 OVERLOADING

SOURCE CODE

```
#include <iostream>
using namespace std;
struct Parcel { double weightKg; bool express; bool insured; };
double charge(int grams) { return grams*0.02; }
double charge(double weightKg, bool express=false, bool insured=false) {
 double amount=weightKg*50.0;
 if(express) amount+=30.0;
 if(insured) amount+=10.0;
 return amount;
}
double charge(const Parcel& p) { return
    charge(p.weightKg,p.express,p.insured); }
int main() {
 cout << charge(500) << '\\n' << charge(2.0) << '\\n'
      << charge(2.0,true) << '\\n' << charge(2.0,true,true) << '\\n'
      << charge(2) << '\\n' << charge(2.0f) << '\\n';
 Parcel p{1.5,false,true}; cout << charge(p) << '\\n';
}
```

TERMINAL OUTPUT

```
10236621002366213023662140236620.042366210023662852366 2
```

Compile: SUCCESS | Run: SUCCESS

Execution status: Compile: SUCCESS | Run: SUCCESS

Q4 Savings Account

C++ PROGRAM / OUTPUT SCREENSHOT — Q4 SAVINGS ACCOUNT

SOURCE CODE

```
#include <iostream>
using namespace std;
class SavingsAccount {
 int accountNumber; double balance; double transactions[10]; int count;
 void record(double amount) {
  if(count<10) transactions[count++]=amount;
  else { for(int i=1;i<10;++i) transactions[i-1]=transactions[i];
     transactions[9]=amount; }
 }
public:
 SavingsAccount(int number,double
    openingBalance):accountNumber(number),balance(openingBalance),count(0){}
 void deposit(double amount){ if(amount>0){balance+=amount;record(amount);} }
 bool withdraw(double amount){ if(amount<=0||amount>balance)return false;
    balance-=amount;record(-amount);return true;}
 void miniStatement() const {
  cout<<"Account: "<<accountNumber<<"\\nBalance: "<<balance<<"\\nRecent
    transactions:\\n";
  for(int i=0;i<count;++i)cout<<transactions[i]<<'\\n';
 }
};
int main(){ SavingsAccount account(1925,1000.0); account.deposit(250);
    account.withdraw(100); account.miniStatement(); }
```

TERMINAL OUTPUT

```
Account: 1925\nBalance: 1150\nRecent transactions:\n25023662-10023662
```

Compile: SUCCESS | Run: SUCCESS

Execution status: Compile: SUCCESS | Run: SUCCESS

Q5 Bus Ticket

C++ PROGRAM / OUTPUT SCREENSHOT — Q5 BUS TICKET

SOURCE CODE

```
#include <iostream>
#include <string>
using namespace std;
class BusTicket {
 static int issued; static const int CAPACITY=3; int seat; string name;
public:
 explicit BusTicket(const string&
    n):seat(0),name(n){if(issued<CAPACITY){++issued;seat=issued;}}
 static int count(){return issued;}
 void print() const { cout<<name<<": "<<(seat ? "Seat
    "+to_string(seat):string("Waiting"))<<'\\n'; }
};
int BusTicket::issued=0;
int main(){
 BusTicket trip[4]={BusTicket("Asha"),BusTicket("Bala"),BusTicket("Chitra"),Bu
    sTicket("Dev")};
 for(const BusTicket& t:trip)t.print();
 cout<<"Issued: "<<BusTicket::count()<<'\\n';
}
```

TERMINAL OUTPUT

```
Asha: Seat 123662Bala: Seat 223662Chitra: Seat 323662Dev: Waiting23662Issued: 323662
```

Compile: SUCCESS | Run: SUCCESS

Execution status: Compile: SUCCESS | Run: SUCCESS

Q6 Time Friend

C++ PROGRAM / OUTPUT SCREENSHOT — Q6 TIME FRIEND

SOURCE CODE

```
#include <iostream>
#include <cstdlib>
using namespace std;
class Time {
 int h,m;
public:
 Time(int hh,int mm):h(hh),m(mm){}
 friend int minutesBetween(const Time&,const Time&);
 friend bool earlier(const Time&,const Time&);
};
int minutesBetween(const Time& a,const Time& b) {
 int first=a.h*60+a.m, second=b.h*60+b.m;
 return abs(first-second);
}
bool earlier(const Time& a,const Time& b) {
 if(a.h!=b.h)return a.h<b.h;
 return a.m<b.m;
}
int main(){
 Time departure(9,45), arrival(13,10);
 cout<<minutesBetween(departure,arrival)<<'
    '<<boolalpha<<earlier(arrival,departure)<<'\\n';
}
```

TERMINAL OUTPUT

```
205 false23662
```

Compile: SUCCESS | Run: SUCCESS

Execution status: Compile: SUCCESS | Run: SUCCESS

Q7 Circular Buffer

C++ PROGRAM / OUTPUT SCREENSHOT — Q7 CIRCULAR BUFFER

SOURCE CODE

```
#include <iostream>
using namespace std;
class History {
 static const int CAP=5;
 int data[CAP], start, count;
public:
 History():start(0),count(0){}
 void record(int value) {
  if(count<CAP){data[(start+count)%CAP]=value;++count;}
  else {data[start]=value;start=(start+1)%CAP;}
 }
 void display() const {
  for(int i=0;i<count;++i) cout<<data[(start+i)%CAP]<<' ';
  cout<<'\\n';
 }
};
int main(){ History h; for(int x=1;x<=7;++x)h.record(x); h.display(); }
```

TERMINAL OUTPUT

```
3 4 5 6 7 23662
```

Compile: SUCCESS | Run: SUCCESS

Execution status: Compile: SUCCESS | Run: SUCCESS

Program Execution Screenshots

The following screenshots show source code and terminal output from programs compiled and executed with g++ -std=c++17 -Wall -Wextra. These are execution evidence panels; if your faculty requires screenshots from a specific IDE, capture those separately on your own computer.

Q1 Call By Reference

C++ PROGRAM / OUTPUT SCREENSHOT — Q1 CALL BY REFERENCE

SOURCE CODE

```
#include <iostream>
using namespace std;
void changeByValue(int a) { a += 10; }
void changeByReference(int& b) { b += 10; }
void changeByPointer(int* c) { if (c != nullptr) *c += 10; }
int& bigger(int& p, int& q) { return (p > q) ? p : q; }
void show(int x, int y = 5, int z = 10) { cout << x + y + z << ' '; }
int main() {
 int a=1, b=2, c=3;
 changeByValue(a); changeByReference(b); changeByPointer(&c);
 cout << a << ' ' << b << ' ' << c << '\n';
 bigger(a,c)=100;
 cout << a << ' ' << c << '\n';
 show(1); show(1,2); show(1,2,3); cout << '\n';
}
```

TERMINAL OUTPUT

```
1 12 13
1 100
16 13 6
```

Compile: SUCCESS | Run: SUCCESS

Execution status: Compile: SUCCESS | Run: SUCCESS

Q1 Concession

C++ PROGRAM / OUTPUT SCREENSHOT — Q1 CONCESSION

SOURCE CODE

```
#include <iostream>
using namespace std;
void applyByValue(long long fee, int percent) {
 fee -= fee * percent / 100;
}
void applyConcession(long long& fee, int percent) {
 fee -= fee * percent / 100;
}
int main() {
 long long hostelFee=45000;
 applyByValue(hostelFee,10);
 cout << "after call by value : " << hostelFee << '\n';
 applyConcession(hostelFee,10);
 cout << "after call by reference: " << hostelFee << '\n';
}
```

TERMINAL OUTPUT

```
after call by value : 45000
after call by reference: 40500
```

Compile: SUCCESS | Run: SUCCESS

Execution status: Compile: SUCCESS | Run: SUCCESS

Q2 Highest Mark

C++ PROGRAM / OUTPUT SCREENSHOT — Q2 HIGHEST MARK

SOURCE CODE

```
#include <iostream>
using namespace std;
int& highestMark(int marks[], int n) {
 int index=0;
 for(int i=1;i<n;++i) if(marks[i]>marks[index]) index=i;
 return marks[index];
}
int main() {
 int marks[]={78,91,84,88};
 highestMark(marks,4)+=2;
 cout << "Highest mark after moderation: " << highestMark(marks,4) << '\n';
}
```

TERMINAL OUTPUT

```
Highest mark after moderation: 93
```

Compile: SUCCESS | Run: SUCCESS

Execution status: Compile: SUCCESS | Run: SUCCESS

Q3 Overloading

C++ PROGRAM / OUTPUT SCREENSHOT — Q3 OVERLOADING

SOURCE CODE

```
#include <iostream>
using namespace std;
struct Parcel { double weightKg; bool express; bool insured; };
double charge(int grams) { return grams*0.02; }
double charge(double weightKg, bool express=false, bool insured=false) {
 double amount=weightKg*50.0;
 if(express) amount+=30.0;
 if(insured) amount+=10.0;
 return amount;
}
double charge(const Parcel& p) { return
    charge(p.weightKg,p.express,p.insured); }
int main() {
 cout << charge(500) << '\n' << charge(2.0) << '\n'
      << charge(2.0,true) << '\n' << charge(2.0,true,true) << '\n'
      << charge(2) << '\n' << charge(2.0f) << '\n';
 Parcel p{1.5,false,true}; cout << charge(p) << '\n';
}
```

TERMINAL OUTPUT

```
10
100
130
140
0.04
100
85
```

Compile: SUCCESS | Run: SUCCESS

Execution status: Compile: SUCCESS | Run: SUCCESS

Q4 Savings Account

C++ PROGRAM / OUTPUT SCREENSHOT — Q4 SAVINGS ACCOUNT

SOURCE CODE

```
#include <iostream>
using namespace std;
class SavingsAccount {
 int accountNumber; double balance; double transactions[10]; int count;
 void record(double amount) {
  if(count<10) transactions[count++]=amount;
  else { for(int i=1;i<10;++i) transactions[i-1]=transactions[i];
     transactions[9]=amount; }
 }
public:
 SavingsAccount(int number,double
    openingBalance):accountNumber(number),balance(openingBalance),count(0){}
 void deposit(double amount){ if(amount>0){balance+=amount;record(amount);} }
 bool withdraw(double amount){ if(amount<=0||amount>balance)return false;
    balance-=amount;record(-amount);return true;}
 void miniStatement() const {
  cout<<"Account: "<<accountNumber<<"\nBalance: "<<balance<<"\nRecent
    transactions:\n";
  for(int i=0;i<count;++i)cout<<transactions[i]<<'\n';
 }
};
int main(){ SavingsAccount account(1925,1000.0); account.deposit(250);
    account.withdraw(100); account.miniStatement(); }
```

TERMINAL OUTPUT

```
Account: 1925
Balance: 1150
Recent transactions:
250
-100
```

Compile: SUCCESS | Run: SUCCESS

Execution status: Compile: SUCCESS | Run: SUCCESS

Q5 Bus Ticket

C++ PROGRAM / OUTPUT SCREENSHOT — Q5 BUS TICKET

SOURCE CODE

```
#include <iostream>
#include <string>
using namespace std;
class BusTicket {
 static int issued; static const int CAPACITY=3; int seat; string name;
public:
 explicit BusTicket(const string&
    n):seat(0),name(n){if(issued<CAPACITY){++issued;seat=issued;}}
 static int count(){return issued;}
 void print() const { cout<<name<<": "<<(seat ? "Seat
    "+to_string(seat):string("Waiting"))<<'\n'; }
};
int BusTicket::issued=0;
int main(){
 BusTicket trip[4]={BusTicket("Asha"),BusTicket("Bala"),BusTicket("Chitra"),Bu
    sTicket("Dev")};
 for(const BusTicket& t:trip)t.print();
 cout<<"Issued: "<<BusTicket::count()<<'\n';
}
```

TERMINAL OUTPUT

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

Compile: SUCCESS | Run: SUCCESS

Execution status: Compile: SUCCESS | Run: SUCCESS

Q6 Time Friend

C++ PROGRAM / OUTPUT SCREENSHOT — Q6 TIME FRIEND

SOURCE CODE

```
#include <iostream>
#include <cstdlib>
using namespace std;
class Time {
 int h,m;
public:
 Time(int hh,int mm):h(hh),m(mm){}
 friend int minutesBetween(const Time&,const Time&);
 friend bool earlier(const Time&,const Time&);
};
int minutesBetween(const Time& a,const Time& b) {
 int first=a.h*60+a.m, second=b.h*60+b.m;
 return abs(first-second);
}
bool earlier(const Time& a,const Time& b) {
 if(a.h!=b.h)return a.h<b.h;
 return a.m<b.m;
}
int main(){
 Time departure(9,45), arrival(13,10);
 cout<<minutesBetween(departure,arrival)<<'
    '<<boolalpha<<earlier(arrival,departure)<<'\n';
}
```

TERMINAL OUTPUT

```
205 false
```

Compile: SUCCESS | Run: SUCCESS

Execution status: Compile: SUCCESS | Run: SUCCESS

Q7 Circular Buffer

C++ PROGRAM / OUTPUT SCREENSHOT — Q7 CIRCULAR BUFFER

SOURCE CODE

```
#include <iostream>
using namespace std;
class History {
 static const int CAP=5;
 int data[CAP], start, count;
public:
 History():start(0),count(0){}
 void record(int value) {
  if(count<CAP){data[(start+count)%CAP]=value;++count;}
  else {data[start]=value;start=(start+1)%CAP;}
 }
 void display() const {
  for(int i=0;i<count;++i) cout<<data[(start+i)%CAP]<<' ';
  cout<<'\n';
 }
};
int main(){ History h; for(int x=1;x<=7;++x)h.record(x); h.display(); }
```

TERMINAL OUTPUT

```
3 4 5 6 7
```

Compile: SUCCESS | Run: SUCCESS

Execution status: Compile: SUCCESS | Run: SUCCESS